23. Singular Value Decomposition

"It had to be $U\Sigma V^T$"

Previous Section:

22. Recommender Systems with Collaborative Filtering

Contents:

Alright. This one tends to intimidate people the moment they see the name. Singular Value Decomposition.

It sounds heavy. It sounds abstract. And at first glance, it feels like one of those topics that belongs strictly in Linear Algebra, far away from anything practical. But that assumption is exactly where things start to get interesting. Because this is not just about breaking a matrix into pieces. This is about understanding *structure*.

Let's start from something simple. When you look at data, what do you actually see?

- A table of numbers
- A matrix filled with values

- Rows and columns that seem, ordinary

But just like in Machine Learning, raw data is never just raw data. There are patterns hidden inside. Relationships. Directions where the data “moves” more strongly than others.

The problem is: those patterns are not obvious.

So the real question becomes: **How do we reveal the structure hidden inside a matrix?**

This is where Singular Value Decomposition comes in. Instead of treating a matrix as a single block of numbers, SVD does something much more insightful:

- It breaks the matrix into three components
- Each component captures a different aspect of the data
- And together, they expose the underlying structure in a way that is far easier to understand and manipulate

So no, this is not Divide and Conquer in the usual sense. This is more like:
Observe → Decompose → Understand

And here is the part that connects it to Machine Learning. Many problems in ML are not about raw data. They are about reducing complexity, extracting meaningful features, removing noise, finding the most important directions in the data

SVD does exactly that. It quietly sits behind techniques like dimensionality reduction, recommendation systems, and even parts of deep learning pipelines.

So while it may look like pure mathematics, it is actually one of the most practical tools you will ever encounter. And once you see what U, Σ, and V^T really represent. You stop seeing matrices as numbers. You start seeing them as **structure waiting to be revealed**.

1. Singular Value Decomposition - The Overview

You see a matrix. Rows. Columns. Numbers. At first, it looks complete. Self-contained. Nothing more to extract. But that is never true. Because a matrix is not just data. It is **relationships compressed into numbers**.

So instead of asking *"What is inside this table?"* → I ask a different question: **"What structure is this table hiding?".** Singular Value Decomposition answers that. Not by simplifying the matrix. Not by approximating it. But by **re-expressing it** in a way that reveals how it was built.

Let's make it concrete. Imagine a table of users and movies.

- Each row → a person
- Each column → a movie
- Each value → a rating

It looks simple. But it isn't. Because behind every number, there are hidden factors:

- Some people prefer action
- Others prefer romance
- Some movies are similar in style
- Some ratings matter more than others

You cannot see these directly. So we decompose. SVD splits the matrix into three parts:

- U → describes the *people*
- Σ → describes the *importance* of each hidden factor
- V^T → describes the *movies*

Not randomly. Not approximately. Precisely. Each component isolates a different perspective of the same data.

So instead of one dense table, you now have:

- Who behaves similarly
- What matters the most
- Which items are related

The workflow is simple in appearance:

- Take the original matrix
- Decompose it into three structured components
- Use those components to understand, reduce, or reconstruct the data

But the meaning behind it is deeper. This is not just splitting. This is **translation**.

Mathematically, we write it as: $A = U\Sigma V^T$

Here is what each part represents:

- U**:** An $m \times m$ orthogonal matrix. Its columns are the **left singular vectors.** They capture how rows (people) relate to hidden patterns
- Σ**:** An $m \times n$ diagonal matrix. Contains the **singular values**, ordered from largest to smallest. They measure how important each pattern is
- V^T**:** The transpose of an n \times n orthogonal matrix. Its columns are the **right singular vectors.** They capture how columns (items) relate to those same patterns

So no, this is not just a formula. It is a statement:

> ***Every matrix can be understood as interactions between two sets of structures, weighted by importance.***

And once you see it that way. You stop reading matrices. You start **interpreting them**.

2. Singular Value Decomposition - An Example

Theory alone is never enough. It tells you *what exists*. But practice shows you **why it matters**. So let's stop talking about matrices in the abstract. Let's actually look at one.

We have a simple dataset:

Name	Movie1	Movie2	Movie3	Movie4
Adam	5	4	2	3
Ben	3	2	3	3
Brad	4	4	3	1
Cindy	1	2	3	1
Declan	1	1	5	1

Five people. Four movies. At first glance, it looks like just ratings. But this is exactly the kind of structure SVD is designed to reveal.

This is our matrix $A = X = \begin{bmatrix} 5 & 4 & 2 & 3 \\ 3 & 2 & 3 & 3 \\ 4 & 4 & 3 & 1 \\ 1 & 2 & 3 & 1 \\ 1 & 1 & 5 & 1 \end{bmatrix}$

Now here is the rule. We are not going to call SVD directly. Because if you do that, you skip the understanding. Instead, we reconstruct it step by step. And no, you are not doing this by hand. That would miss the point. We let computation handle the weight. We focus on **what each step reveals**.

Step 1 — Compute $A^T A$

This compares columns with columns.

So now the question becomes:

- How similar is Movie 1 to Movie 2?
- Which movies tend to be rated similarly?

This is no longer about people. This is about **item relationships**.

```
import numpy as np

A = np.array([[5, 4, 2, 3],
              [3, 2, 3, 3],
              [4, 4, 3, 1],
              [1, 2, 3, 1],
              [1, 1, 5, 1]])

ATA = A.T @ A
print(f"A^T * A:\n", ATA)
```

```
A * A^T:
[[52 45 39 30]
[45 41 37 25]
[39 37 56 26]
[30 25 26 21]]
```

💡 The choice between AA^T or A^TA depends heavily on what you want to focus on:

- Using AA^T, I observe the **row space**. This means I am comparing people against each other. The eigenvectors extracted from this matrix become the **left singular vectors** U, because they describe how individuals align with hidden patterns.
- Using A^TA, I shift the perspective to the **column space**. Now I am comparing movies against each other. The eigenvectors here become the **right singular vectors** V, because they capture how items relate through shared structure.

Since the goal is to understand how movies relate, the correct lens is A^TA. This is not a different method. It is the same decomposition, viewed from the side that reveals what I need.

Step 2 — Eigenvalues of $A^TA \rightarrow A^TAv = \lambda v$

Eigenvalues tell us something critical: **Which patterns matter the most.**

- Large eigenvalues → strong structure
- Small eigenvalues → weak or noisy structure

```
# Eigenvalues and eigenvectors
eigenvalues, eigenvectors = np.linalg.eig(ATA)
print("Eigenvalues:\n", eigenvalues)
print("Eigenvectors:\n", eigenvectors)
```

```
Eigenvalues:
[147.87287667  17.6397502   0.45716781  4.03020531]
Eigenvectors:
[[-0.56906083 -0.47058785 -0.67431945 -0.00318136]
[-0.50926195 -0.2810943   0.62837252 -0.51651354]
[-0.54473182  0.82827853 -0.11805972 -0.05730474]
[-0.34653901 -0.11613676  0.36946334  0.85435345]]
```

Step 3 — Right Singular Vectors

This is where people usually get confused.

We use eigenvectors... but they are not just vectors.

They are **directions of behavior**.

Each vector represents a pattern like:

- “Prefers high ratings overall”
- “Likes Movie 3 more than others”

```
V = eigenvectors
print("V:\n", V)
```

V:
[[-0.56906083 -0.47058785 -0.67431945 -0.00318136]
[-0.50926195 -0.2810943 0.62837252 -0.51651354]
[-0.54473182 0.82827853 -0.11805972 -0.05730474]
[-0.34653901 -0.11613676 0.36946334 0.85435345]]

Step 4 — Compute Left Singular Vectors: $u_i = \frac{1}{\sigma_i} A v_i$

Now we move from **patterns** back to **people**. Each u_i tells us: “How strongly does each person align with this pattern?”

```
sigma = np.sqrt(eigenvalues)
U = []
for i in range(len(sigma)):
    if sigma[i] > 1e-10:
        ui = (A @ V[:, i]) / sigma[i]
        U.append(ui)

U = np.array(U).T
print("U:\n", U)
```

U:
[[-0.57658389 -0.51647263 0.02094737 0.18255697]
[-0.44402846 0.03868436 -0.01775141 0.67175592]
[-0.51758805 -0.15191293 -0.24921332 -0.69554897]
[-0.29344008 0.31807901 0.88400052 -0.17622051]
[-0.34115286 0.77942783 -0.39456545 0.02397707]]

Step 5 — Compute the Singular Values $\Sigma \rightarrow \sigma_i = \sqrt{\lambda_i}$

The singular values do not come from nowhere. They are derived directly from the eigenvalues we computed earlier.

```
sigma = np.sqrt(eigenvalues)
Sigma = np.diag(sigma)
print("Sigma:\n", Sigma)
```

```
Sigma:
[[12.1602992  0.         0.         0.        ]
 [ 0.         4.19997026 0.         0.        ]
 [ 0.         0.         0.67614186 0.        ]
 [ 0.         0.         0.         2.00753713]]
```

Each singular value measures how strong a pattern is and how much that pattern contributes to reconstructing the data. Larger value → more important; Smaller value → less important.

Step 6 — Final Equation: $A = U\Sigma V^T$

Now everything comes together.

- U → how people align with patterns
- Σ → how important each pattern is
- V^T → how movies are structured

```
A_reconstructed = U @ Sigma @ V.T
print("A_reconstructed:\n", A_reconstructed)
```

```
A_reconstructed:
[[5. 4. 2. 3.]
 [3. 2. 3. 3.]
 [4. 4. 3. 1.]
 [1. 2. 3. 1.]
 [1. 1. 5. 1.]]
```

If the process is correct, this reconstruction should closely match the original matrix.

This is the full decomposition. Not a shortcut. Not a black box. A complete breakdown of structure into: **alignment** → **importance** → **relationship**

3. Singular Value Decomposition - Applications

Singular Value Decomposition (SVD) is one of the most important techniques in linear algebra and data analysis.

By decomposing a matrix into simpler components, SVD allows complex data to be analyzed, compressed, and reconstructed efficiently.

Because of its ability to reveal hidden structures and patterns, SVD is widely used in mathematics, engineering, and machine learning applications.

- Homogeneous Linear Equations
- Curve Fitting Problems
- Digital Signal Processing: SVD can be used to analyze signals and filter noise.
- Image Processing: SVD is used for image compression and denoising by preserving the most significant singular values while reducing less important information.

Summary

Singular Value Decomposition is more than just a linear algebra technique; it is also a foundational concept in machine learning and data science. By breaking data into meaningful components, SVD helps uncover patterns, reduce dimensionality, and simplify complex datasets.

At its core, SVD reflects one of the main goals of machine learning: learning and representing patterns from data in the most efficient and meaningful way possible.

References

https://www.geeksforgeeks.org/data-science/singular-value-decomposition-svd/

https://www.ibm.com/think/topics/singular-value-decomposition

Next Section:

- 24. Linear Discriminant Analysis